Functional Programming Languages

What is a functional programming language?

Functional programming languages mean “function” in the same way you learn in mathematics. They define the outputs as a mathematical function of the inputs.

Recursive-function definition and invocation, without side-effects

Side Effect: function or expression modifies state outside its local environment or interacts with the outside world beyond returning a value

- “Pure” functional languages do not allow side effects at all
- Based on Alonzo Churches 1930’s “lambda calculus”
  - Functions need not be named
  - Lambda Calculus restrict Functions to only allow one input (or parameter)
    - Most Functional PLs aren’t so restrictive.
  - Contrast to a “Turing Machine”

What makes a programming language functional?

1. First-class function values and higher-order functions
    a. First class means a value that can be passed or returned from a function
    b. Higher order functions mean that they can take functions as arguments and return function(s)
2. Extensive polymorphism
3. List types and operators
4. Structured function returns
5. Constructors for structured objects
6. Garbage Collection

Functional Programming Innovations

Recursion (recursion)

First Class Functions (functions are objects)

Closure (function state)

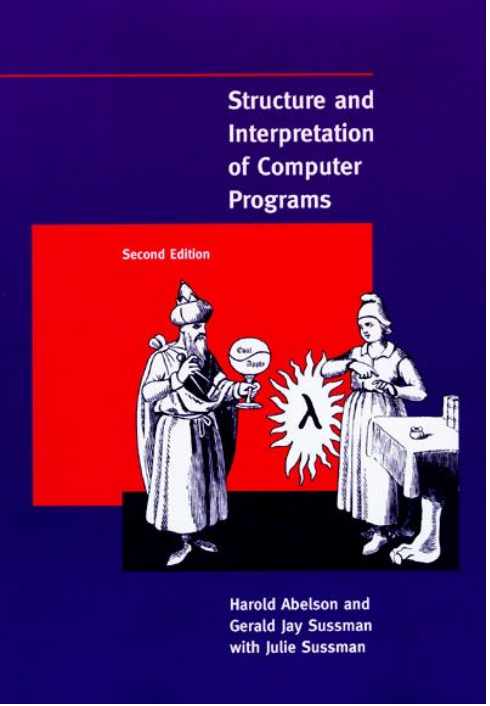
Lambda expressions (anonymous - having no name)

Normal order evaluation (lazy evaluation)

Syntax-rules (true macros)

Tail call optimization (efficient recursion)

Memoization (expression caching)


Structure and
Interpretation
of Computer
Programs
Second Edition
λ
Harold Abelson and
Gerald Jay Sussman
with Julie Sussman

Scheme



Origins

Scheme and Common Lisp are modern dialects of the original Lisp, which was designed by John McCarthy at MIT in 1958.

- Only Fortran is only, and just then by 1 year.
- Lisp is for “**Li**st **Pr**ocessing”

A subset of lisp is “pure” but over time some Imperative features were built in. To ward programmers off from using them, the side-effecting functions are preceded with a ‘!’

Otherwise, Lisp is nearly unchanged since its origins

Scheme

Scheme and Lisp use extremely simple syntax. C's grammar is hundreds of lines long. Scheme's grammar can be written in just 6 (almost).

About the language:

- Dynamic types
- Static scope
- Higher order functions
- Soooo much more.

What is a program?

A program in Scheme (and lisp) is a parenthesized sequence of symbolic expressions.

An **expression** is a statement that always produces a value

A **literal** evaluates to itself (3 evaluates to 3 for instance)

A **symbol** evaluates to its defined value

A parenthesized sequence evaluates to the result of calling the first and passing the values of the rest. Polish notation, AKA prefix form.

Expressions which do not follow this rule are called *special-forms*

Simple examples

- echo.scm
- sum.scm

What's with the parenthesis?

- Actually, not much different from what you are used to.
- Function definition and call in java

```
int foo(a,b) { body } -> foo(a, b)
```

- Function call in Scheme

```
(define (foo a b) body) -> (foo a b)
```

- There are actually fewer characters, because the missing commas

Functions

So… define is a function, not a keyword?

What about…

+ or -

In a simple sense, there are only functions

- (not strictly true, but good enough for now)

More examples

- factorial.scm
- echo-lambda.scm

```
(lambda (x) (+ x 4))
```

The procedure of an argument x that adds x and 4.

Everything is a list

Constructs in scheme are all lists. When we call sum, we need to provide the parameter as a list, and not as a function call.

```
(sum ‘(1 2 3))
```

Is shorthand for

```
(sum (quote 1 2 3))
```

Lists

Lists are fundamental (the language came from one called List Processing)

Lists are a data structure composed of Pairs and Atoms. (`car` refers to the first element of a pair, and `cdr` the remainder. They are named after registers in the IBM 701)

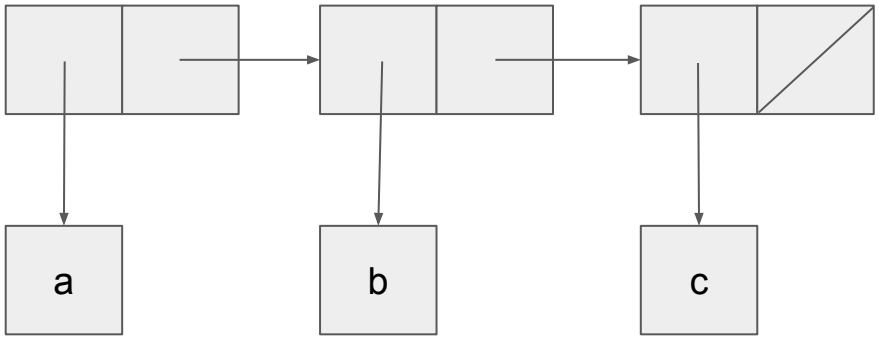
Programs are also lists! Therefore, Scheme is homoiconic

Lists can be built with cons. Actually, all of the following produce equivalent lists

- `‘(a b c), (quote (a b c)), (list ‘a ‘b ‘c),`
  `(cons ‘a (cons ‘b (cons ‘c ‘())))`

Structure of a list

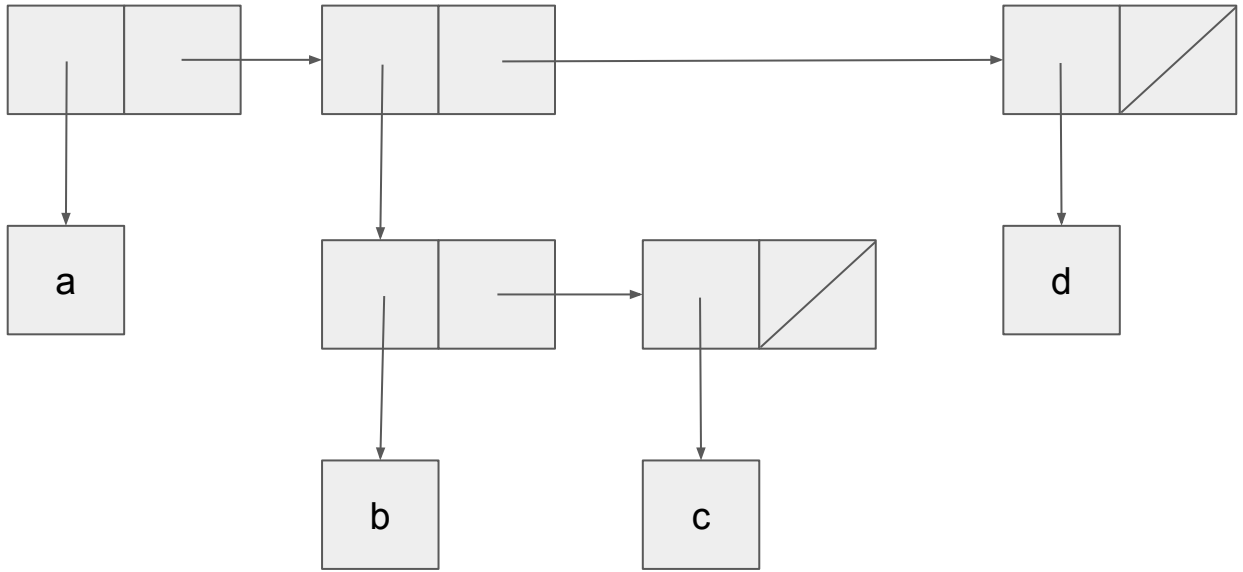
`(a b c)`



Also equivalent to `(a b c . '())`

Structure of a list

```
(a (b c) d)
```



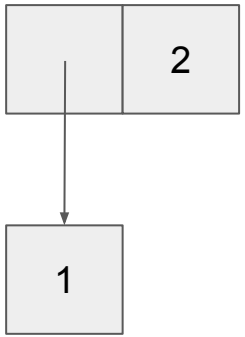
Building a List

Scheme provides some built in utilities to help construct lists.

`(cons)`

`(append)`

Dot syntax is sometimes seen if you build lists incorrectly. `(cons 1 2)` produces



Alternatively: `(1 . 2)`

Building a List II

```
(cons 1 2) -> (1 . 2)       ; incorrect

(pair (cons 1 2))           ; #t

(pair (car (cons 1 2)))     ; #f

(pair (cdr (cons 1 2)))     ; #f


(cons 1 '()) -> (1) or (1 . '()) ; correct
```

Computation in Scheme

Now we have:

1. Data structure (lists)
2. Model of computation

So we can compute things on data.

Try:

list-length.scm

For you: x.scm

Why learn functional programming

Another technique to add to your toolkit

Simplify certain sets of problems

Change the way you think about programming

An example from CS321

Project 4 required students to traverse the tree in two different ways.

1. Traverse to print
2. Traverse to collect for the unit tester

Functional programming can simplify this by generalizing the action taken:

1. Traverse to “do something”

Action.java

Advanced functional programming

Here are some concepts from functional programming that will be returning this semester.

- Tail call optimization
- Memoization
- Currying (named for Haskell Curry, also namesake of Haskell PL)